

Simple Web Application Threat Model

Owner:

Reviewer:

Contributors:

Date Generated: Fri Jun 05 2026

Executive Summary

High level system description

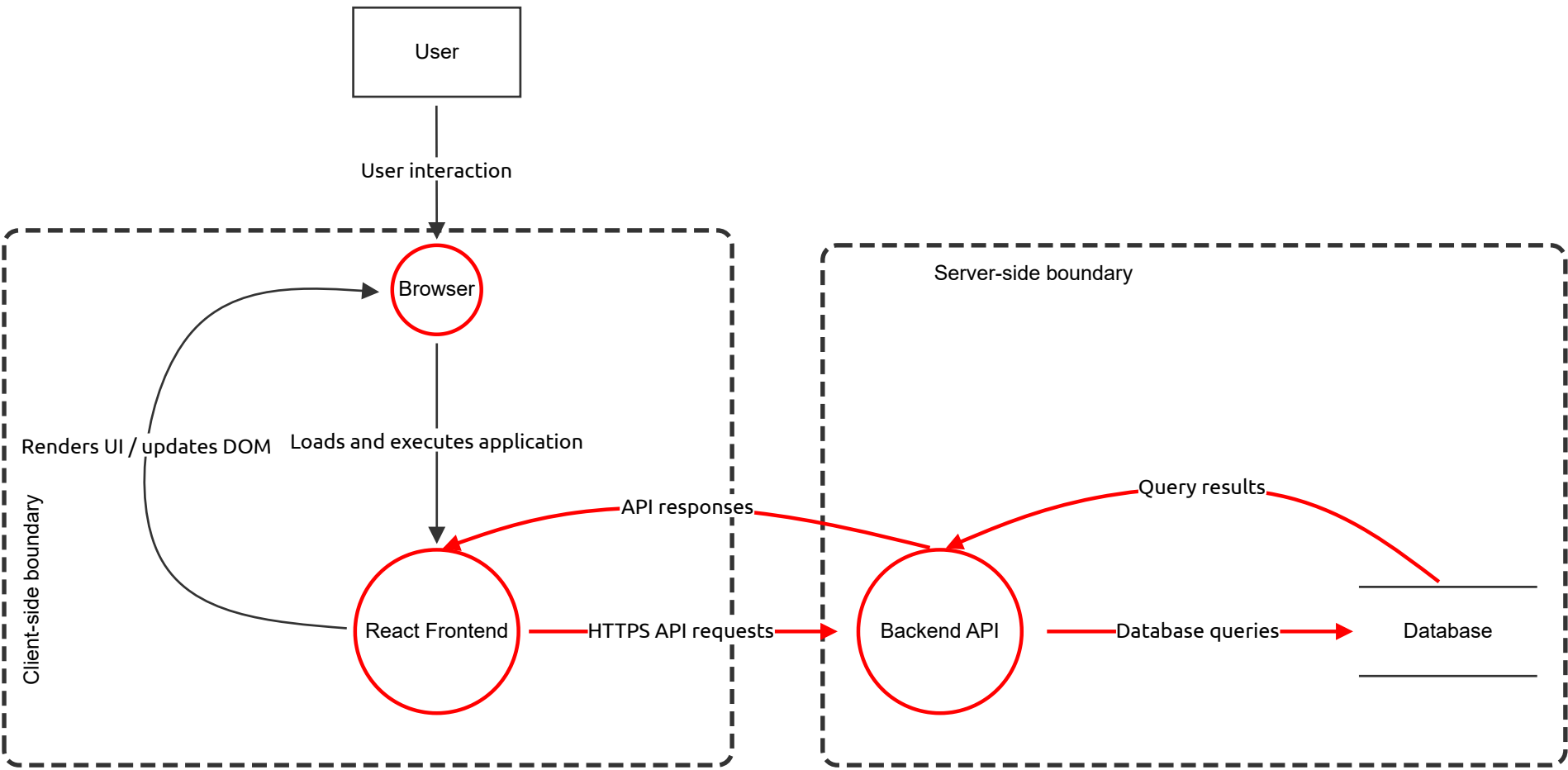
Not provided

Summary

Total Threats	8
Total Mitigated	0
Total Open	8
Open / Critical Severity	0
Open / High Severity	5
Open / Medium Severity	3
Open / Low Severity	0

Web Application Architecture

Data flow between the user, React frontend, backend API and database.



Web Application Architecture

User (Actor)

Properties:

Number	Title	Type	Severity	Status	Score	Description	Mitigations
--------	-------	------	----------	--------	-------	-------------	-------------

React Frontend (Process)

Description: Validation:
Use SAST and code review to identify console.log(), console.info(), console.debug(), and other debug statements that may expose tokens, personal data, API responses, internal identifiers, or implementation details.

Use DAST as a supporting check by reviewing the browser console and runtime errors in the production build.

Validation status:
Not tested. Hypothetical threat model.

Properties:

Number	Title	Type	Severity	Status	Score	Description	Mitigations
6	Sensitive Information Exposed in Browser Console	Information disclosure	Medium	Open		The React application may expose sensitive information through console.log(), console.info(), console.debug(), or unhandled error messages. This may include API responses, tokens, user data, internal identifiers, feature flags, or implementation details.	Remove debug logging from production builds. Never log tokens, credentials, personal data, or complete API responses. Use a controlled logging utility, environment-based log levels, and review production bundles for debug statements.

Backend API (Process)

Properties:

Number	Title	Type	Severity	Status	Score	Description	Mitigations
2	Broken Object-Level Authorization	Elevation of privilege	High	Open		The backend may allow a user to access or modify another user's object by changing an object identifier in the request.	Validation: Verify that the backend identifies the authenticated user from a validated session cookie or access token and does not trust a client-supplied userId. Review and test whether every object request checks that the authenticated user has permission to access or modify the requested resource. Test modified object identifiers while keeping the same authenticated session to confirm that one user cannot access another user's data. Mitigation: Verify object ownership and permissions on every request. Do not rely on hidden frontend controls or client-supplied user identifiers.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
5	Missing Audit Logging	Repudiation	Medium	Open		Sensitive actions may not be recorded with enough information to determine who performed them and when.	Validation: Review whether sensitive backend operations create structured audit events containing the authenticated user ID, action performed, target resource, timestamp, request or correlation ID, outcome, source IP where appropriate, and relevant application context. Verify that audit logs cannot be modified by ordinary users and that security-relevant events can be searched and traced across related requests. Validation status: Not tested. Hypothetical threat model. Mitigation: Create structured audit logs for security-sensitive operations. Record who performed the action, what action was attempted, which resource was affected, when it occurred, and whether it succeeded or failed. Include useful context such as: authenticated user ID, timestamp in UTC, action and target resource, request or correlation ID, success or failure result, source IP where reliable, user agent where useful. Do not log passwords, full session tokens, access tokens, sensitive personal data, payment details, or complete request bodies by default. Protect logs from tampering and define retention and access-control rules.

Database (Store)

Properties:

Number	Title	Type	Severity	Status	Score	Description	Mitigations
--------	-------	------	----------	--------	-------	-------------	-------------

Browser (Process)

Description: An attacker who steals a session cookie or access token may impersonate a legitimate user.

Validation:

Verify that session cookies use the Secure, HttpOnly and appropriate SameSite attributes. Check that tokens are not stored in localStorage unless there is a justified design reason, sessions expire correctly, and logout invalidates the active session.

Validation status:

Not tested. Hypothetical threat model.

Properties:

Number	Title	Type	Severity	Status	Score	Description	Mitigations
3	Session or Token Theft	Spoofing	High	Open		An attacker who steals a session cookie or access token may impersonate a legitimate user.	Use Secure, HttpOnly and appropriate SameSite cookie settings. Protect against XSS, rotate tokens, expire sessions and invalidate compromised sessions.

User interaction (Data Flow)

Properties:

Number	Title	Type	Severity	Status	Score	Description	Mitigations
--------	-------	------	----------	--------	-------	-------------	-------------

Loads and executes application (Data Flow)

Properties:

Number	Title	Type	Severity	Status	Score	Description	Mitigations
--------	-------	------	----------	--------	-------	-------------	-------------

Renders UI / updates DOM (Data Flow)

Properties:

Number	Title	Type	Severity	Status	Score	Description	Mitigations
--------	-------	------	----------	--------	-------	-------------	-------------

API responses (Data Flow)

Description: Validation:
Review API responses and compare returned fields with what the React application actually uses. Check whether the API exposes unnecessary personal data, internal identifiers, roles, permissions, security-related fields, or records belonging to other users.

Validation status:
Not tested. Hypothetical threat model.

Properties:

Number	Title	Type	Severity	Status	Score	Description	Mitigations
4	Excessive Data Returned by API	Information disclosure	High	Open		The API may return fields or records that the current user does not need or is not authorised to access.	Return only required fields, enforce server-side authorisation and use response schemas or DTOs to prevent accidental data exposure.

HTTPS API requests (Data Flow)

Description: Validation:
Review whether the backend independently identifies the authenticated user, verifies their permissions, validates all client-controlled values, and returns only the data and actions they are authorised to access. Test modified requests to confirm that changing userId, role, price, or object identifiers does not bypass server-side controls.

Validation status:
Not tested. Hypothetical threat model.

Properties:

Number	Title	Type	Severity	Status	Score	Description	Mitigations
1	Modified API Request	Tampering	High	Open		A user can modify frontend-generated API requests before they reach the backend, including values such as userId, role, price or object identifiers.	Apply a zero-trust approach to all frontend input. Treat every request field as attacker-controlled. Enforce authentication, authorisation, input validation, ownership checks, and business rules on the backend. Return only the data required for the authenticated user and requested operation.

Database queries (Data Flow)

Description: Validation:
Review the backend data-access code and verify that:

untrusted input is never concatenated into query strings, prepared statements or parameterised queries are used consistently, ORM raw query usage is identified and reviewed,

inputs with a limited set of valid values use strict allowlists,
database accounts follow least privilege,
SQL or database error details are not exposed to users.

Use SAST and manual code review to identify unsafe query construction. Where authorised, test relevant endpoints with controlled injection payloads and compare application responses.

Validation status:
Not tested. Hypothetical threat model.

Properties:

Number	Title	Type	Severity	Status	Score	Description	Mitigations
7	Injection in Database Query	Tampering	High	Open		User-controlled input may be included in database queries without safe parameterisation or strict query construction. This could allow an attacker to alter query logic, access unauthorised data, modify records, bypass authentication, or trigger database errors.	Use parameterised queries or prepared statements. For NoSQL databases, enforce strict input types and schemas, reject operator-shaped input, and avoid passing user-controlled objects directly into query filters. Apply least-privilege permissions to the database account.

Query results (Data Flow)

Description: Validation:
review selected fields and query scope,
check whether raw database objects are returned,
verify authorisation before data retrieval,
compare returned data with what the API/frontend actually needs.

Properties:

Number	Title	Type	Severity	Status	Score	Description	Mitigations
8	Sensitive Data Returned from Database	Information disclosure	Medium	Open		Database queries may return more records or fields than the current user is authorised to access.	Enforce authorisation before and after data retrieval, limit selected fields, use scoped queries, and avoid returning raw database objects directly to the client.